

Approximation Algorithms & Complexity

1. Define Deterministic and Non-Deterministic Algorithms.

Deterministic Algorithm

A deterministic algorithm **always computes the correct answer**.

Example: *Linear and Binary Search, Sorting problems.*

Non-Deterministic Algorithm

A non-deterministic algorithm can “**guess**” the right answer or solution.

Example: Graph Coloring, Travelling Salesman.

2. Write a Non-Deterministic Algorithm for Binary Search.

Non-Deterministic Algorithm for Searching

```
Search_NP( key )
{
    array list[100];
    i = check_array( key );
    if (list[i] == key)
        Searching Found at index i
    else
        Searching Not Found
}
```

Annotations:

- $O(1)$: No idea about how it works (points to `i = check_array( key );`)
- $O(1)$ (points to `if (list[i] == key)`)
- $O(1)$ (points to `Searching Found at index i`)
- $O(1)$ (points to `Searching Not Found`)

No comparing, searching

**So, Total Time Complexity of above algorithm is $O(1)$.
But it is Non-Deterministic Algorithm.**

3. What is an NP-Complete Problem?

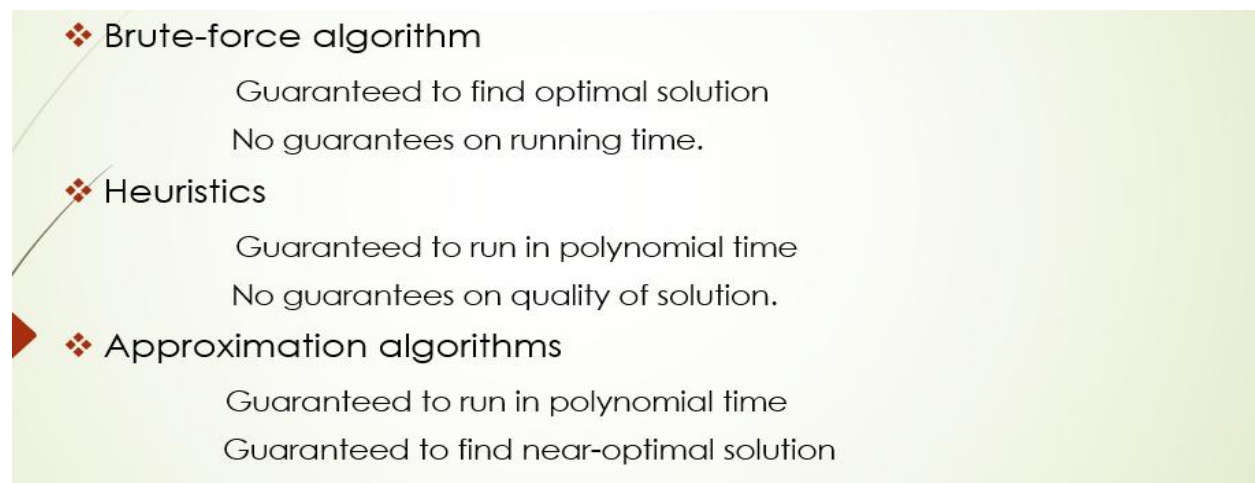
An **NP-Complete problem** is a problem for which there is **no polynomial-time algorithm to find an optimal solution**. These problems are very difficult to solve.

Examples:

- TSP
 - 0/1 Knapsack
 - Graph Coloring
-

4. How Can NP-Complete Problems Be Solved? / 2. How do you address NP-Complete Problems?

NP-Complete problems can be solved in several ways:



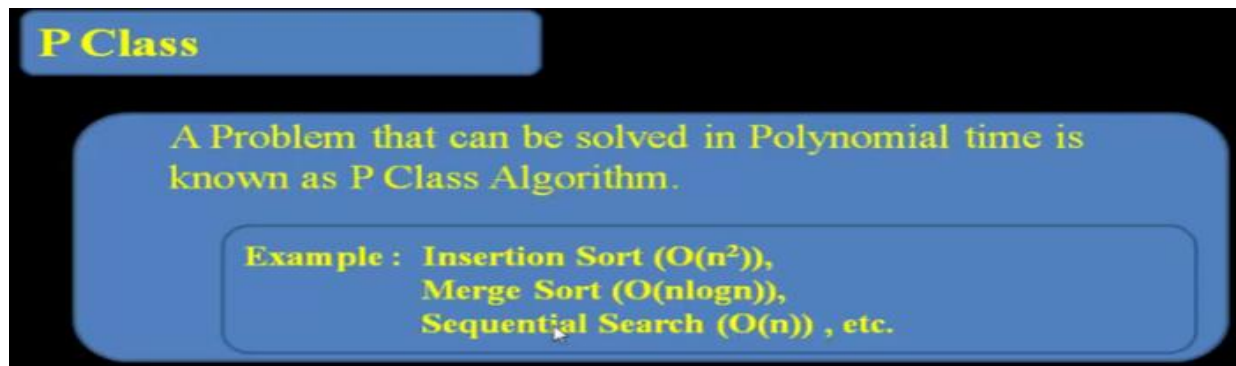
5. What is a Complexity Class? Role of Complexity Classes in Computational Complexity.

A **complexity class** is a set of computational problems having related **resource-based complexity**. The main resources are **time and memory**.

Role:

- Classifies computational problems.
 - Helps compare problem difficulty.
 - Shows the resources required to solve problems.
-

6. Define P, NP, NP-Complete and NP-Hard Complexity Classes with Examples.



P Class

A Problem that can be solved in Polynomial time is known as P Class Algorithm.

Example : Insertion Sort ($O(n^2)$), Merge Sort ($O(n \log n)$), Sequential Search ($O(n)$) , etc.

NP Class

NP contains decision problems that can be solved in polynomial time using a **non-deterministic algorithm**.

Examples: Graph Coloring, TSP, Satisfiability.

Define NP-Hard and NP-Complete Problem

NP-Hard:

A problem is called **NP-Hard** if it is at least as hard as every problem in NP.

Examples: TSP, Graph Coloring, K-means.

NP-Complete:

A problem is **NP-Complete** if it is both **NP** and **NP-Hard**.

Examples: TSP, Vertex Cover, Hamiltonian Cycle.

7. Compare NP-Complete and NP-Hard Complexity Classes with Related Examples.

NP-Complete	NP-Hard
Problems are in NP.	Problems are at least as hard as NP problems.
Solution can be verified in polynomial time.	May not be verifiable in polynomial time.

NP-Complete	NP-Hard
Examples: TSP, Knapsack, Graph Coloring	Examples: TSP, Graph Coloring, K-means

8. Formulate the Bin Packing Problem as an Approximation Problem. State the First-Fit 2-Approximation Algorithm with Example.

Bin Packing is an optimization problem where items of different sizes are packed into bins of fixed capacity, with the goal of **minimizing the number of bins**.

Approximation Problem:

Find a solution that uses a number of bins **close to the minimum**.

First-Fit 2-Approximation Algorithm

Algorithm 2 FIRSTFIT(S)

```

B  $\rightarrow \emptyset$ 
for  $i \in \{1, \dots, n\}$  do
  if  $size(i) \leq size(b)$  for some bin  $b \in B$  then
     $size(b) \leftarrow size(b) - size(i)$ 
  else
     $B \leftarrow B \cup \{b'\}$ , where  $size(b') = 1 - size(x_i)$ 
  end if
end for
return B

```

Example

Bin capacity = **10**

Items = **6, 4, 5, 5, 3, 7**

- Bin 1 $\rightarrow 6 + 4 = 10$
- Bin 2 $\rightarrow 5 + 5 = 10$
- Bin 3 $\rightarrow 3 + 7 = 10$

Therefore, **3 bins are used**.

Approximation ratio:

$$FF(I) \leq 2OPT(I)$$

9. List the Applications of Bin Packing Problem.

applications are:

1. **Loading containers like trucks**
 2. **Placing data on multiple disks**
 3. **Job scheduling**
 4. Packing advertisements in fixed-length **radio/TV station breaks**
 5. Creating **file backups**
 6. Technology mapping in **FPGA semiconductor chip design**.
-

10. What is Approximation Ratio?

The **approximation ratio** shows how close the solution obtained by an approximation algorithm is to the **optimal solution**.

For a minimization problem:

$$\frac{C}{C^*} \leq \rho(n)$$

Where:

- C = cost of approximate solution
 - C^* = optimal cost
 - $\rho(n)$ = approximation ratio
-

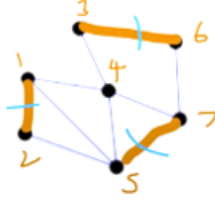
11. Prove: $|C| = 2|M| \leq 2|C^*|$

Given:

- C = a vertex cover
- M = a maximal matching
- C^* = an optimal vertex cover

Matching: In graph theory, a matching or independent edge set in a graph is a set of edges without common vertices.

Maximal Matching: A matching becomes a maximal matching, if no more edges can be added to it.



- ✓ $C = \{1, 2, 5, 7, 3, 6\}$
- ✓ $M = \{(1, 2), (5, 7), (3, 6)\}$
- ✓ $|C| = 6$
- ✓ $|M| = 3$
- ✓ $|C| = 2 \cdot |M|$

Step 1

For every edge in maximal matching M , both endpoints are selected into C .

Therefore,

$$|C| = 2 |M|$$

For the example:

$$M = \{(1, 2), (5, 7), (3, 6)\}$$

So,

$$|M| = 3$$

and

$$C = \{1, 2, 5, 7, 3, 6\}$$

Thus,

$$|C| = 6 = 2 |M|$$

Step 2

Since M is a matching, an optimal vertex cover C^* must contain at least one endpoint of every edge in M .

Therefore,

$$|C^*| \geq |M|$$

Multiplying both sides by 2:

$$2 |C^*| \geq 2 |M|$$

Since,

$$|C| = 2 |M|$$

Therefore,

$$\boxed{|C| \leq 2 |C^*|}$$

Hence proved.

12. Using APPROX-VERTEX-COVER Approximation Algorithm, Solve the Vertex Cover Problem with an Example.

Using Approximate-Max-Cut algorithm, solve the max-cut problem with an example.

APPROX-MAXCUT(G)

1. $C \leftarrow \emptyset$
2. $E' \leftarrow G.E$
3. **while** $E' \neq \emptyset$
4. let (u,v) be an arbitrary edge of E'
5. $C \leftarrow C \cup \{u,v\}$
6. remove from E' every edge incident on either u or v
7. **return** C



Initially, Given input,

$$E' = \{(1,2), (1,3), (1,4), (2,3), (2,5), (3,4), (3,6), (4,5), (4,7), (5,6), (5,7), (6,7)\}$$

1st iteration

let $(1,2)$ edge pick as an arbitrary edge.

$$C = \{1, 2\}$$

$$E' = \{(3,4), (3,6), (4,5), (4,7), (5,6), (5,7), (6,7)\}$$

2nd iteration

let $(5,6)$ be the arbitrary edge

$$C = \{1, 2, 5, 6\}$$

$$E' = \{(3,4), (4,7)\}$$

3rd iteration

let $(3,4)$ be the

$$C = \{1, 2, 5, 6, 3, 4\}$$

$$E' = \emptyset$$

Example

Initially:

$$E' = \{(1,2), (1,5), (1,4), (2,5), (3,4), (3,6), (4,5), (4,7), (5,7), (6,7)\}$$

Step 1

Select (1,2):

$$C = \{1, 2\}$$

Remaining:

$$E' = \{(3,4), (3,6), (4,5), (4,7), (5,7), (6,7)\}$$

Step 2

Select (5,7):

$$C = \{1, 2, 5, 7\}$$

Remaining:

$$E' = \{(3, 4), (3, 6)\}$$

Step 3

Select (3,6):

$$C = \{1, 2, 5, 7, 3, 6\}$$

Now,

$$E' = \emptyset$$

Therefore,

$$C = \{1, 2, 5, 7, 3, 6\}$$

and

$$|C| = 6$$

The corresponding maximal matching is:

$$M = \{(1, 2), (5, 7), (3, 6)\}$$

So,

$$|M| = 3$$

And

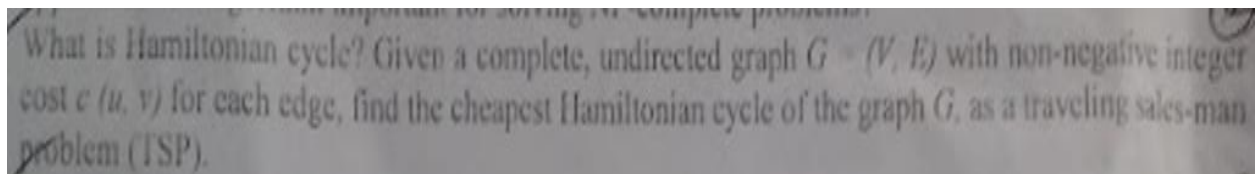
$$|C| = 2|M| = 6$$

3. Why is Approximation Algorithm Important for solving NP -Complete problem?

Approximation algorithms are important because:

- NP-Complete problems are difficult to solve exactly.
 - Brute-force methods may require very large time.
 - Approximation algorithms run in **polynomial time**.
 - They provide a solution that is **close to the optimal solution**.
 - They are useful for large practical problems.
-

4.



A **Hamiltonian cycle** is a cycle in a graph that **visits every vertex exactly once** and returns to the starting vertex.

TSP Formulation

Given a complete, undirected graph:

$$G = (V, E)$$

where each edge (u, v) has a non-negative cost $c(u, v)$.

TSP finds the shortest possible tour that visits every vertex exactly once and returns to the starting vertex.

So, TSP is an **optimization problem**: minimize the total cost of the Hamiltonian cycle.

what is Reduction ?

Reduction means converting one problem into another problem so that solving the second problem helps us solve the first problem.

In **NP-Complete problems**, reduction is used to show that one problem is at least as hard as another problem.

Simple example:

$$\text{Problem } A \rightarrow \text{Problem } B$$

If we can solve **B**, then we can solve **A**.

Formal Definition of Approximation Algorithm

An **approximation algorithm** is a polynomial-time algorithm that finds a **near-optimal solution** to an optimization problem and provides a **guaranteed bound** on how far the solution is from the optimal solution.

Formally, let:

- $A(I)$ = solution produced by the approximation algorithm
- $A^*(I)$ = optimal solution
- $\rho(n)$ = approximation ratio

For a **minimization problem**:

$$\boxed{A(I) \leq \rho(n)A^*(I)}$$

For a **maximization problem**:

$$A(I) \geq \frac{A^*(I)}{\rho(n)}$$

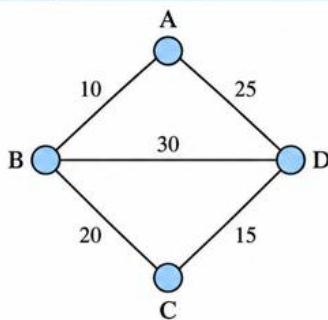
1. Hamiltonian Cycle

† A Hamiltonian cycle is a cycle in a graph that visits every vertex exactly once and returns to the starting vertex.

2. Travelling Salesman Problem (TSP)

Given a complete, undirected graph $G = (V, E)$, where each edge (u, v) has a non-negative integer cost $c(u, v)$, the Travelling Salesman Problem (TSP) asks to find a Hamiltonian cycle with the minimum total cost.

3. Example



Complete undirected graph G with 4 cities and edge costs.

Consider the Hamiltonian cycle:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

This route visits every city exactly once and returns to A, so it is a Hamiltonian cycle.

Edge costs are given as:

Edge	Cost
A - B	10
B - C	20
C - D	15
D - A	25

Total cost of this cycle:

$10 + 20 + 15 + 25 = 70$

Among all possible Hamiltonian cycles, the one having the smallest total cost is the solution to the TSP.

In one line for exam: "TSP finds the minimum-cost Hamiltonian cycle in a complete weighted graph."

1. When Asymptotic Notations are used to analyze the algorithms? Distinguish $O(N^2)$ and $O(2^N)$ with examples. (1+2=3)

When used:

Asymptotic notations are used to analyze how an algorithm's running time or memory grows as the input size (n) increases. They measure efficiency independent of hardware, ignoring constant factors and lower-order terms.

Distinguishing $O(N^2)$ and $O(2^N)$:

Feature	$O(N^2)$ (Quadratic)	$O(2^N)$ (Exponential)
Growth	Grows moderately (polynomial time).	Grows extremely fast (beyond polynomial time).

Feature	$O(N^2)$ (Quadratic)	$O(2^N)$ (Exponential)
Efficiency	Efficient for small/medium inputs.	Computationally hard; impractical for large inputs.
Cause	Usually caused by two nested loops.	Usually caused by checking all subsets of a set.

Examples:

- $O(N^2)$: Finding the closest pair of points among n points by comparing every point with every other point (two nested loops).
- $O(2^N)$: Finding the maximum independent set in a graph by checking every possible subset of nodes (there are 2^n subsets).

2. Why is asymptotic notation important in algorithm analysis and design? Demonstrate and explain the growth of different functions.

Importance:

- **Hardware Independent:** Evaluates algorithm efficiency regardless of processor speed or compiler.
- **Scalability:** Predicts performance as input size grows toward infinity.
- **Simplification:** Reduces complex equations (e.g., $3n^2 + 5n + 12$) to a single dominant term (n^2) for easy comparison.
- **Guarantees:** Provides upper bounds (worst-case) and lower bounds (best-case/fundamental limits).

Growth of Different Functions (Slowest to Fastest):

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < n!$$

- $O(1)$: Constant (e.g., array lookup).
- $O(\log n)$: Logarithmic (e.g., Binary Search).
- $O(n)$: Linear (e.g., single loop).
- $O(n \log n)$: Linearithmic (e.g., Merge Sort).
- $O(n^2)$: Quadratic (e.g., Bubble Sort).

- $O(2^n)$: Exponential (e.g., brute-force subset problems).
-

3. Define asymptotic notation. What is the goal of asymptotic notation? Demonstrate and explain the asymptotic tight bound.

Definition:

Asymptotic notation describes the limiting behavior of an algorithm's time or space complexity as the input size (n) approaches infinity. It discards low-order terms and constant factors to focus strictly on growth rate.

Goal:

To measure running time in a way that is independent of hardware, insensitive to constant factors, and reflects the algorithm's intrinsic efficiency.

Asymptotic Tight Bound (Θ):

Represented by **Theta** (Θ). It bounds the function from both above and below.

- **Definition:** $T(n) = \Theta(f(n))$ if there exist positive constants c_1, c_2, n_0 such that:

$$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n) \quad \text{for all } n \geq n_0$$

This means $T(n) = O(f(n))$ AND $T(n) = \Omega(f(n))$.

- **Demonstration:** Let $T(n) = 7n^2 + 5$. Show $T(n) = \Theta(n^2)$.
 - Upper: $7n^2 + 5 \leq 12n^2$ for $n \geq 1$ ($c_2 = 12$).
 - Lower: $7n^2 + 5 \geq 7n^2$ for $n \geq 0$ ($c_1 = 7$).
 - Since $7n^2 \leq 7n^2 + 5 \leq 12n^2$, $T(n) = \Theta(n^2)$.
-

4. What is the goal of asymptotic notation? Demonstrate and explain the asymptotic upper bound.

(Goal is same as Question 3)

Asymptotic Upper Bound (O):

Represented by **Big-O (O)**. It guarantees the maximum time an algorithm will take (worst-case).

- **Definition:** $T(n) = O(f(n))$ if there exist positive constants c and n_0 such that:

$$T(n) \leq c \cdot f(n) \quad \text{for all } n \geq n_0$$

- **Demonstration:** Let $T(n) = 2n + 3$. Show $T(n) = O(n)$.
 - Find c such that $2n + 3 \leq c \cdot n$.
 - Try $c = 5$: $2n + 3 \leq 5n \Rightarrow 3 \leq 3n \Rightarrow 1 \leq n$.
 - So, for all $n \geq 1$, $2n + 3 \leq 5n$. Thus, $T(n) = O(n)$.
(Note: $T(n)$ is also $O(n^2)$, but $O(n)$ is the tightest and most useful bound).

- i. What is meant by computational complexity? Which is the best algorithm for optimization? 7
- ii. Why optimization algorithm is used? Where can optimization be used? 7

i. What is meant by computational complexity? Which is the best algorithm for optimization?

Computational Complexity:

Best Algorithm for Optimization:

There is no single "best" algorithm; it depends on the problem.

For a linear regression model, the best algorithm used is **Gradient Descent**.

ii. Why optimization algorithm is used? Where can optimization be used?

Why Optimization Algorithms are Used:

They are mathematical methods used to find the **best solution** (maximum or minimum) to a problem. They work by:

- **Minimizing** a cost/loss function.
- **Maximizing** a reward/utility function.

The goal is to find the values of variables that give the best (optimum) outcome.

Where Optimization Can Be Used:

- **Gradient-based** (e.g., Gradient Descent) for ML and Deep Learning.
- **Heuristic/Metaheuristic** (e.g., Genetic Algorithm) for AI and Scheduling.
- **Linear Programming** (e.g., Simplex) for Business and Supply Chains.
- **Constrained** (e.g., Lagrange Multipliers) for Engineering and Physics.
- **Convex** (e.g., Quadratic Programming) for ML and Control Systems.